

Skip – List
–Delete Element
–Time Complexity Analysis

Program :

```
void deleteElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Find the node, keeping track of pointers that point to it
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    current = current->forward[0];

    // If the target node is found
    if (current != NULL && current->key == key) {
        // Rewire the pointers to bypass the deleted node
        for (int i = 0; i <= lst->level; i++) {
            if (update[i]->forward[i] != current) {
                break; // If a level doesn't point to current, higher levels
won't
                // either
            }
            update[i]->forward[i] = current->forward[i];
        }

        // Free memory using free()
        free(current->forward);
        free(current);

        // Reduce the maximum level of the list if layers become empty
```

```

        while (lst->level > 0 && lst->header->forward[lst->level] ==
NULL) {
            lst->level--;
        }
        cout << "Successfully deleted key " << key << endl;
    } else {
        cout << "Key " << key << " not found. Cannot delete." << endl;
    }
}

...
case 2:
    cout << "Enter key to delete: ";
    cin >> key;
    deleteElement(list, key);
    break;

```

Time Complexity Analysis

1. Best Case Time Complexity: $O(1)$

The absolute best – case scenario occurs when the element to be deleted is exactly at the front of the list, or the list is empty or the key is smaller than the first element.

The Search Phase

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

- **The Search Phase:** The horizontal **while** loop condition (**current** → **forward[i]** → **key < key**) fails immediately. The program ducks the horizontal traversal and simply drops straight down the **for** loop to Level 0. Because **lst** → **level** is capped by **MAX_LEVEL** (a hardcoded constant), this vertical drop takes **O(1)** time.

1. "The list is empty"

If we try to delete an element from a Skip List that has no nodes, the header's pointers just point to **NULL**.

- **The Code:** The **while** loop checks **current** → **forward[i]** ≠ **NULL**.
- **The Result:** Because the next node is **NULL**, the condition instantly fails on the very first check at every single level. The pointer never takes a step forward.

2. "The key is smaller than the first element"

Imagine the first actual node in the list has a key of 50, but you ask the program to delete 20.

- **The Code:** The **while** loop checks **current** → **forward[i]** → **key < key**. It compares **50 < 20**.
 - ☐ Level 3: header → forward[3] points to 50. (Checks: 50 < 20 is False)
 - ☐ Level 2: header → forward[2] points to 50. (Checks: 50 < 20 is False)
 - ☐ Level 1: header → forward[1] points to 50. (Checks: 50 < 20 is False)
 - ☐ Level 0: header → forward[0] points to 50. (Checks: 50 < 20 is False)
- **The Result:** Because 50 is not less than 20, the condition instantly fails on the very first check. The program realizes the target number should be to its left, meaning it isn't in the list at all. It never takes a step forward.

3. "The element is exactly at the front"

Imagine the very first node in the list has a key of 30, and we ask the program to delete 30.

- **The Code:** Again, the while loop only moves forward if $current \rightarrow forward[i] \rightarrow key < key$. It compares $30 < 30$.
- **The Result:** Because 30 is exactly equal to 30, the strict less than condition fails! The search pointer realizes it is already looking directly at the target node, so it refuses to move to the right.

The Rewiring Phase

$[lst \rightarrow level \leq MAX_LEVEL]$

```
for (int i = 0; i <= lst->level; i++) {
    if (update[i]->forward[i] != current) {
        break; // If a level doesn't point to current, higher levels
won't
        // either
    }
    update[i]->forward[i] = current->forward[i];
}
```

- **The Rewiring Phase:** If the target is found, the algorithm loops to bypass the deleted node using $update[i] \rightarrow forward[i] = current \rightarrow forward[i]$. Because a single node's height is probabilistically bounded by a small constant, and the loop includes a break statement to exit early when higher levels don't point to **current**, this takes $O(1)$ time.

The Height Adjustment Phase

```
while (lst->level > 0 && lst->header->forward[lst->level] ==  
NULL) {  
    lst->level--;  
}
```

- **The Height Adjustment Phase:** The algorithm uses a while loop to decrement the list's overall level ($lst \rightarrow level --$) if the top layers become empty after deletion. Because the level is strictly capped, this takes $O(1)$ time.
- **Result:** $O(1)$ Search + $O(1)$ Rewiring + $O(1)$ Adjustment = $O(1)$.

Worst Case Scenario

The Root Cause: The Inherited Structure

The worst – case scenario mathematically devolves to $O(n)$ due to one of two severe structural failures that occurred when the list was built:

The All Tails Scenario (The Flat List): Every node previously inserted into the list flipped Tails on its first try and stopped at Level 0. The list has zero upper – level express lanes and physically resembles a standard singly linked list.

The All Heads Scenario (The Traffic Jam): Every node previously inserted flipped Heads repeatedly, promoting every single node to the absolute maximum level. The top floor becomes just as crowded as the ground floor, clogging the express lanes.

[This was discussed during time – complexity description of
Inserting Element.]

If the algorithm attempts to delete a node located at the very end of either of these broken structures, here is exactly how the three phases of the deletion code perform.

Phase 1: The Search Phase (The $O(n)$ Bottleneck)

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

To delete a node, the algorithm must first execute a normal search to find it. This normal search relies on upper – level express lanes to skip over nodes. However, if the physical structure of the Skip List is severely unbalanced, the search loses its skipping ability and degrades to $O(n)$.

This structural failure happens in two physical layouts:

- ***Structural Failure 1 (The Flat List):*** *Imagine the Skip List was built entirely of short nodes that only exist at Level 0. Because the maximum level of the list is 0, the vertical for loop executes exactly once. The inner while loop is forced to do all the heavy lifting. If the target node is at the very end of the list, the pointer must physically step through all n nodes one by one on the ground floor.*
- ***Structural Failure 2 (The Clogged Top Floor):*** *Imagine the list was built entirely of tall nodes, meaning every single node exists at the maximum level (e. g., Level 3). The normal search correctly starts at the top floor. However, because every node has a pointer on Level 3, the pointer cannot skip over anything. The while loop is forced to evaluate every single node horizontally across the crowded top floor.*

Result: *In either degenerate physical structure, the normal search algorithm is forced to perform n sequential horizontal comparisons to reach the end of the list. This phase degrades to $O(n)$.*

Phase 2: Pointer Rewiring & Deallocation

$[lst \rightarrow level \leq MAX_LEVEL]$

```
for (int i = 0; i <= lst->level; i++) {
    if (update[i]->forward[i] != current) {
        break; // If a level doesn't point to current, higher levels
won't
        // either
    }
    update[i]->forward[i] = current->forward[i];
}
```

Once the $O(n)$ search finally reaches the target node, the algorithm must rewire the pointers to bypass it.

- **The Code:** The algorithm uses a for loop running from 0 to $lst \rightarrow level$ to bridge the gaps: $update[i] \rightarrow forward[i] = current \rightarrow forward[i]$.
- **In the All Tails Flat List:** The deleted node only exists at Level 0. On the very next iteration ($i = 1$), the condition $if (update[i] \rightarrow forward[i] \neq current)$ evaluates to True, the break statement fires, and the loop terminates immediately.
- **In the All Heads Traffic Jam:** The deleted node exists on every level up to MAX_LEVEL . The for loop must run MAX_LEVEL times to rewire every single floor.
- **Result:** Because MAX_LEVEL is strictly bounded by a mathematical cap (e.g., a hardcoded 3, or logarithmically capped at $\log_2 n$), therefore this loop can never scale to n . The pointer rewiring, along with the $free(current \rightarrow forward)$ and $free(current)$ commands, executes in a maximum of $O(\log n)$ or $O(1)$ time.

Phase 3: Global Height Adjustment

```
while (lst->level > 0 && lst->header->forward[lst->level] ==  
NULL) {  
    lst->level--;  
}
```

Finally, the algorithm cleans up the list's global height ceiling.

- **The Code:** A **while** loop checks $lst \rightarrow header \rightarrow forward[lst \rightarrow level] == NULL$ and shrinks the list via $lst \rightarrow level --$ if a floor becomes completely empty.
- **The Execution:** Even if deleting the final node leaves multiple top floors completely empty (such as deleting the final tall node in an "All Heads" list), the **while** loop can only drop the level down from **MAX_LEVEL** to 0.
- **Result:** Because it is bounded by the **MAX_LEVEL** cap, this phase takes a maximum of **$O(\log n)$** or **$O(1)$** time.

Final Calculation

To calculate the absolute Worst Case Time Complexity, we sum the mathematical bounds of the three phases:

- Phase 1 (Search): **$O(n)$**
- Phase 2 (Rewiring): **$O(\log n)$ or $O(1)$**
- Phase 3 (Adjustment): **$O(\log n)$ or $O(1)$**

Because the linear **n** term massively dominates the constant and logarithmic terms as the list grows towards infinity, the smaller terms are dropped.

Total Worst Case Time Complexity = $O(n)$

3. Average Case Time Complexity: $O(\log n)$

The expected time complexity is determined by breaking the *deleteElement* algorithm down into its three sequential phases.

Phase 1: The Search Phase

- The function uses a vertical for loop and a horizontal *while* loop to populate the *update* array with the memory addresses of predecessor nodes.
- As established in William Pugh's backward walk derivations, perfectly balanced coin flips ($p = 0.5$) allow the pointer to systematically skip halves of the list. The expected time to locate the node to be deleted is $O(\log n)$.

Phase 2: Pointer Rewiring & Deallocation

- To physically detach the node, the code iterates upward from 0 to *lst* $\rightarrow$ *level*.
- Inside the loop, the algorithm checks *if* (*update*[*i*] $\rightarrow$ *forward*[*i*] $\neq$ *current*) and breaks early if higher levels do not point to the deleted node. This highly optimized loop only runs for the specific height of the deleted node, not the maximum height of the entire list.
- Because the expected height of any single node is a tiny constant (e. g., an average of 2 pointers when $p = 0.5$), the rewiring loop and subsequent *free()* commands execute in $O(1)$ expected time.

Phase 3: Global Height Adjustment

- After the node is removed from memory, a while loop checks *lst* $\rightarrow$ *header* $\rightarrow$ *forward*[*lst* $\rightarrow$ *level*] $==$ *NULL*. If the deleted node was the only tall node keeping the list's maximum level high, the code safely shrinks the global list height (*lst* $\rightarrow$ *level* $--$).
- Because deleting a single node will rarely shrink the list by more than 1 or 2 levels, this cleanup loop executes in $O(1)$ expected time.

Final Calculation:

To find the average case, we sum the mathematical bounds of the three phases:

- *Search: $O(\log n)$ + Rewiring: $O(1)$ + Adjustment: $O(1)$*
 - *Total Expected Time Complexity = $O(\log n)$*
-